

Experiment 8

Support Vector Machine

Aim: To implement Support Vector Machine classifiers using linear and non-linear kernels for classification tasks, and to evaluate their performance on linearly and non-linearly separable datasets.

Theory:

1. Introduction

Support Vector Machines (SVMs) are supervised machine learning algorithms used for classification and regression tasks. They were formally introduced by Cortes and Vapnik in 1995 as part of the statistical learning theory based on the principle of Structural Risk Minimization (SRM).

The main objective of SVM is to find an optimal decision boundary that separates different classes in a dataset with the maximum possible margin. Instead of simply minimizing classification error, SVM attempts to maximize the distance between the separating boundary and the closest data points of each class, which improves the model's generalization ability.

In a feature space, SVM identifies a boundary known as a hyperplane that divides the data points into different categories. Among many possible hyperplanes, the algorithm selects the one that maximizes the margin between classes, making the classifier robust and less sensitive to noise.

2. Hyperplane

The hyperplane divides the feature space into two regions corresponding to different classes. For a two-dimensional dataset, the hyperplane is a straight line, whereas in three dimensions it becomes a plane. In higher-dimensional spaces, it is referred to as a hyperplane.

SVM does not choose just any separating hyperplane; instead, it selects the optimal hyperplane that maximizes the distance from the nearest training points of both classes. This property makes SVM robust and improves its ability to generalize to unseen data.

the SVM seeks a separating hyperplane defined by:

$$w^T x + b = 0$$

where w is the weight vector which is normal to the hyperplane and b is the bias term.
Note - An optimal hyperplane is the one that maximizes the margin between the classes.

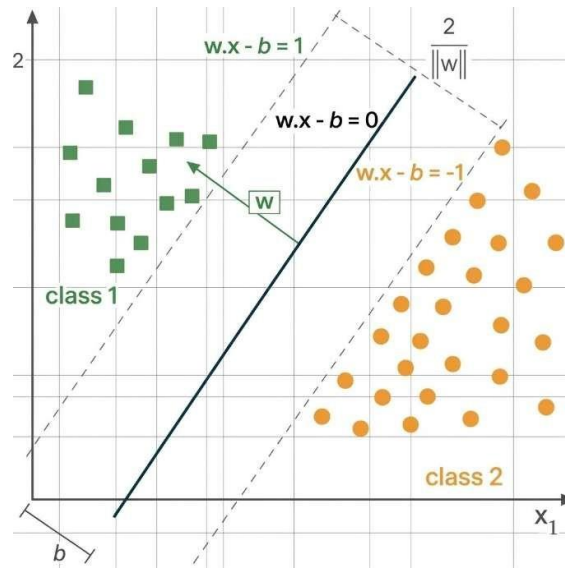


Figure 1: Linear Support Vector Machine (SVM): Maximum Margin Classifier

As shown in Figure 1, the Support Vector Machine (SVM) separates two classes of data points in a two-dimensional feature space defined by x_1 and x_2 . The green points represent Class 1, while the orange points represent Class 2. The solid line $w \cdot x - b = 0$ represents the optimal separating hyperplane that divides the two classes. The dashed lines $w \cdot x - b = 1$ and $w \cdot x - b = -1$ denote the margin boundaries, and the distance between them is referred to as the maximum margin. The vector w is perpendicular to the hyperplane and determines its orientation.

3. Support Vectors

Support vectors are the data points that lie closest to the separating hyperplane. These points play a critical role in defining the position and orientation of the decision boundary. The distance between the hyperplane and these closest data points is known as the margin. Only these boundary points influence the formation of the hyperplane, while other points that lie farther away do not significantly affect the model. Because the hyperplane depends directly on these points, the algorithm is called a Support Vector Machine.

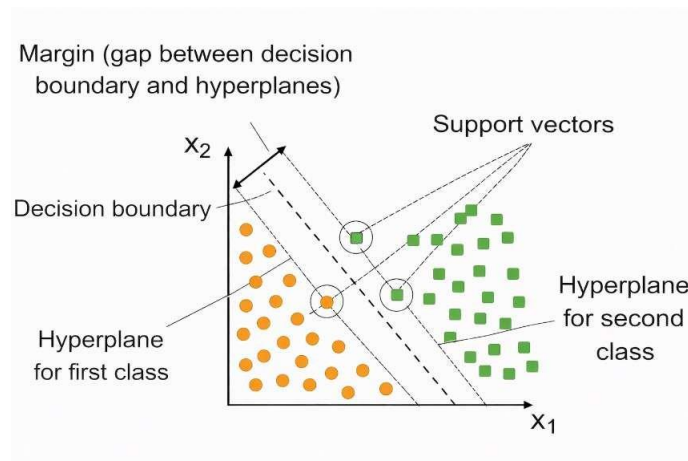


Figure 2: Support Vector Machine Classification with Maximum Margin and Support Vectors

As shown in Figure 2, the Support Vector Machine (SVM) separates two classes of data points in a two-dimensional feature space represented by x_1 and x_2 . The orange circular points represent the first class, while the green square points represent the second class. The central dashed line represents the decision boundary that divides the two classes, and the two parallel dashed lines on either side denote the hyperplanes that define the margin.

The region between these hyperplanes represents the margin, which is the distance between the decision boundary and the nearest data points. The data points that lie closest to these margin boundaries are known as support vectors, and they play a crucial role in determining the optimal separating hyperplane. SVM aims to maximize this margin to achieve improved classification performance and better generalization.

4. Margin Maximization

This optimization problem attempts to minimize the magnitude of the weight vector while satisfying the classification constraints. By minimizing $\|w\|^2$, the margin between the two classes becomes larger. A larger margin leads to a classifier that is less sensitive to noise and better able to generalize to unseen data.

The margin maximization problem can be formulated as:

$$\min_{w,b} \frac{1}{2} \|w\|^2$$

with constraint,

$$y_i(w \cdot x_i + b) \geq 1$$

for all training samples (x_i, y_i) . This formulation ensures that the separating hyperplane lies as far as possible from the closest data points, thereby enhancing the classifier's ability to generalize to unseen data.

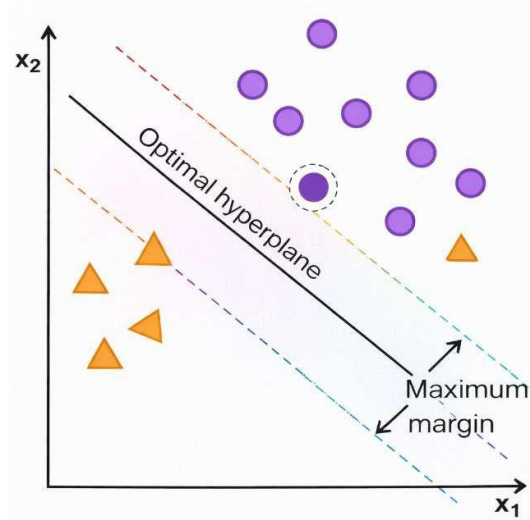


Figure 3: Optimal Hyperplane and Maximum Margin in Support Vector Machines

As shown Figure 3, the Support Vector Machine (SVM) separates two classes of data points in a two-dimensional feature space defined by the axes x_1 and x_2 . The purple circular points represent one class, while the orange triangular points represent the other. The solid slanted line labelled “Optimal hyperplane” acts as the decision boundary that divides the two classes.

The two parallel dashed lines on either side of this hyperplane denote the margin boundaries. The distance between these boundaries is referred to as the maximum margin, which SVM aims to maximize for improved generalization. The data points that lie closest to these margin lines are known as support vectors, and they are critical in determining the position and orientation of the optimal hyperplane.

5. Kernel Trick

In many real-world problems, the data is not linearly separable in the original feature space. SVM solves this problem using the kernel trick, which implicitly maps the data into a higher-dimensional feature space where linear separation becomes possible. Instead of explicitly computing the transformation into the higher-dimensional space, kernel functions compute the inner product of transformed feature vectors directly. This makes the computation efficient even when the dimensionality of the transformed space is very large.

The kernel trick allows SVM to construct non-linear decision boundaries in the original input space while still maintaining the mathematical formulation of a linear classifier in the transformed feature space.

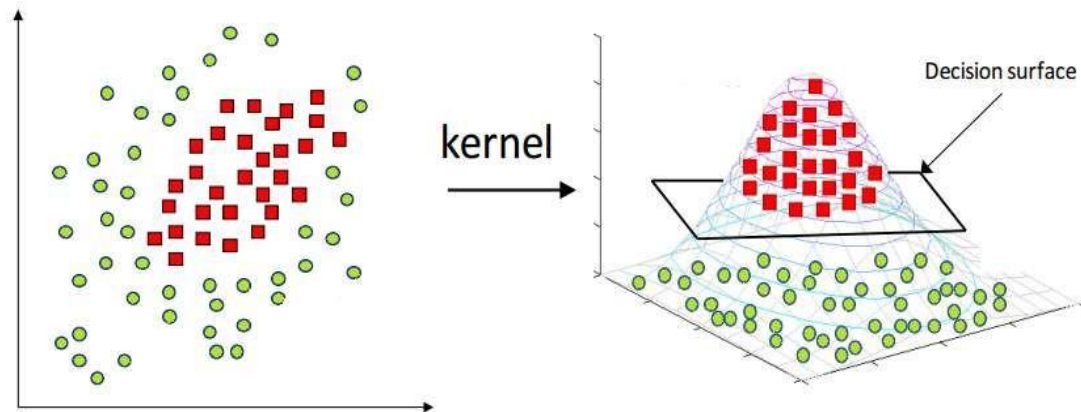


Figure 4: kernel Trick: Mapping Data to Higher-Dimensional Space in Support Vector Machines

As shown in the above Figure 4, the diagram explains how Kernel Support Vector Machines (Kernel SVM) work. On the left side, the data points belong to two different classes: green circles and red squares. In this two-dimensional space, the data is not linearly separable, meaning a straight line cannot properly divide the two classes.

To solve this problem, a kernel function is applied, which transforms the data into a higher-dimensional space. This transformation is illustrated on the right side of the figure, where the data is mapped into a three-dimensional space. In this higher dimension, the data becomes linearly separable, and a decision surface (hyperplane) can be used to separate the two classes effectively.

Thus, the kernel trick allows SVM to handle complex, non-linear classification problems by mapping the data into a higher-dimensional feature space where a linear separator can be found.

Common Kernel functions include:

- **Linear Kernel:** The linear kernel is suitable when the data is approximately linearly separable.

$$K(x_i, x_j) = x_i \cdot x_j$$

- **Radial Basis Function (RBF) Kernel:** The RBF kernel maps data into an infinite-dimensional feature space and is highly effective for capturing complex, non-linear relationships.

$$K(x_i, x_j) = e^{\frac{-|x_i - x_j|^2}{2\sigma^2}}$$

The variance (σ^2) controls the influence of individual training samples. Higher values lead to tighter decision boundaries.

The RBF kernel is particularly effective and is commonly used for modelling complex, non-linear relationships due to its radially localised response characteristics.

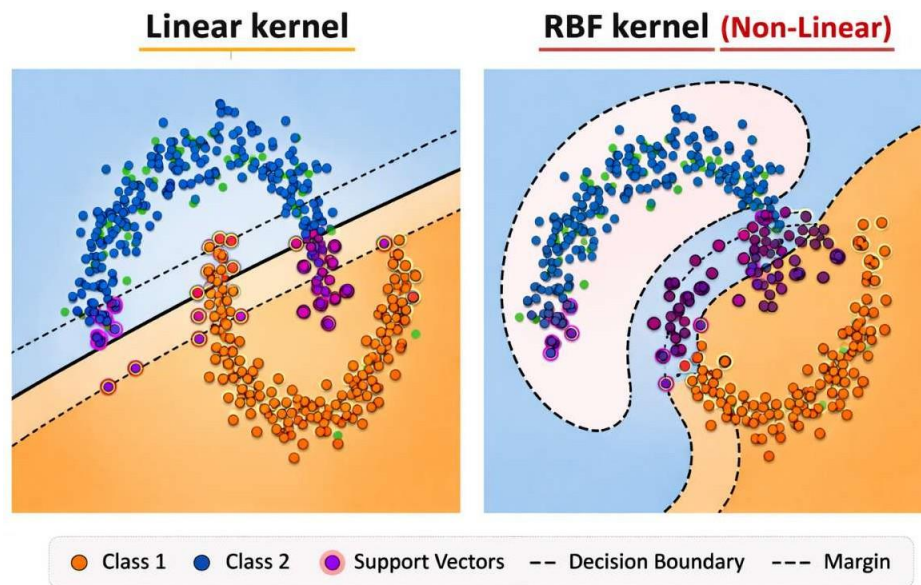


Figure 5: Comparison of Linear and RBF Kernels in Support Vector Machines

The figure compares Support Vector Machine classification using Linear and RBF kernels. In the Linear kernel, a straight-line decision boundary separates the data, suitable for linearly separable patterns. In contrast, the RBF kernel produces a curved boundary, allowing it to capture complex, non-linear relationships and achieve better separation of overlapping classes.

6. L1 Regularization in SVM

Regularization plays an important role in controlling model complexity and preventing overfitting. In Support Vector Machines, regularization is controlled through the parameter C , which balances margin maximization and classification error.

L1 regularization introduces sparsity in the model by encouraging some weight coefficients to become zero. This helps in feature selection and improves model interpretability. By penalizing the

absolute magnitude of the weights, L1 regularization reduces the influence of less important features and helps create simpler models.

7. Algorithm

Step 1: Given training data with two classes (+1 and -1)

Step 2: Find hyperplane: $w \cdot x + b = 0$

w = weight vector (perpendicular to hyperplane)

b = bias term

Step 3: Define margin constraints:

For class +1 points: $w \cdot x_i + b \geq +1$

For class -1 points: $w \cdot x_i + b \leq -1$

Combined: $y_i(w \cdot x_i + b) \geq 1$

Step 4: Margin width = $2/\|w\|$

Step 5: Optimization problem:

Minimize: $(1/2)\|w\|^2$

Subject to: $y_i(w \cdot x_i + b) \geq 1$ for all i

Step 6: Solve using Lagrange multipliers:

Convert to dual form

Find α_i values for each training point

Support vectors are points where $\alpha_i > 0$

Step 7: For non-linear data, apply Kernel Trick:

RBK Kernel: $K(x, x') = \exp(-\gamma \|x - x'\|^2)$

Polynomial Kernel: $K(x, x') = (x \cdot x' + c)^d$

Maps data to higher dimension where linear separation is possible

Step 8: For prediction:

Calculate: $f(x) = \sum(\alpha_i \times y_i \times K(x_i, x)) + b$

If $f(x) \geq 0$ then it belongs to Class +1, otherwise, it belongs to class -1

8. Merits of Support Vector Machines

- **Good generalization performance**

SVMs focus on maximizing the margin between classes, which helps the model perform well on unseen data and reduces overfitting, especially in high-dimensional datasets.

- **Works well for both linear and non-linear data**

By using different kernel functions such as Linear and RBF, SVMs can handle simple linearly separable data as well as complex non-linear patterns effectively.

- **Uses only important data points**

The model depends mainly on support vectors, which are the most critical data points near the decision boundary. This makes the classifier efficient and robust.

9. Demerits of Support Vector Machines

- **High training time for large datasets**

SVM training can be slow and computationally expensive when the dataset is very large, particularly when non-linear kernels are used.

- **Sensitive to parameter selection**

The performance of SVM strongly depends on choosing the right kernel and hyper-parameters. Incorrect values can lead to poor classification results.

- **Harder to interpret results**

Unlike simpler models such as Linear Regression or Decision Trees, especially SVMs with non-linear kernels do not provide clear insights into how individual features might affect predictions.

Procedure

The objective of this part of the experiment is to implement a Support Vector Machine (SVM) classifier on a real-world dataset to study linear class separability. The Wine Recognition dataset is used, where selected chemical attributes are employed to classify three types of wines: **Barolo**, **Grignolino**, and **Barbera**. This part focuses on understanding linear decision boundaries, margin maximization, and the effect of feature selection on classification accuracy.

Part 1: Linear SVM on Wine Recognition Dataset

1. Import *numpy* and *pandas* for numerical computation and data handling, *matplotlib* and *seaborn* for data visualization, and *sklearn* for pipelines, model calls and evaluations.
2. Dataset Loading and Description:
 - Load the Wine Recognition Dataset using *load_wine()* from *sklearn.datasets*
 - The dataset consists of:
 - 178 instances
 - 13 numerical chemical attributes
 - 3 target classes representing wine cultivars:
 - **Class 0: Barolo**
 - **Class 1: Grignolino**
 - **Class 2: Barbera**
3. Select only two features: **Flavanoids** and **Color Intensity** as using two features enables direct 2D visualization and gives clear interpretation of class separability
4. Exploratory Data Analysis (EDA):
 - Plot scatter plots of **Flavanoids** vs **Color Intensity** with class-wise colour coding.
 - Plot histograms for each selected feature across different classes.
 - Analyse overlap between classes to assess linear separability.
5. Data Preprocessing:
 - a) Define feature matrix X using the selected attributes and target vector y using class labels.

- b) Split the dataset into training and testing sets using an 80:20 ratio.
- c) Apply standardization using *StandardScaler()* to normalize feature distributions.
- 6. Model Training:
 - Train a Support Vector Machine classifier with a linear kernel on the training data.
 - The model attempts to find an optimal hyperplane that maximizes the margin between classes.
- 7. Model Evaluation (Linear SVM):
 - Evaluate the trained model on the test dataset using Accuracy, Precision, Recall and F1-score metrics and visualize using Confusion Matrix.
 - Analyse misclassifications and class-wise performance.
- 8. Decision Boundary Visualization: Plot the linear decision boundary along with support vectors. Observe how a straight hyperplane separates the wine classes in feature space.

Part 2: Non-Linear SVM on Two Moons Dataset

The Two Moons dataset is used to demonstrate the limitations of linear classifiers and the necessity of kernel-based transformations. By applying an RBF kernel, this part highlights how kernel selection enables flexible decision boundaries and improves classification performance on non-linearly structured data.

- 1. Import *numpy*, *matplotlib*, and relevant modules from *sklearn*.
- 2. Generate the Two Moons dataset using *make_moons()*
- 3. Exploratory Data Analysis (EDA):
 - Plot a scatter plot of the dataset to visualize the non-linear class distribution.
 - Observe the curved structure that motivates the use of kernel methods.
- 4. Data Preprocessing:
 - a) Define X as the 2D coordinates of the points and y as the binary class labels.
 - b) Split the dataset into training and testing sets using an 80:20 ratio.
 - c) Apply *StandardScaler()* to improve kernel performance and convergence.
- 5. Train an SVM classifier using the Radial Basis Function (RBF) kernel.
- 6. Evaluate the model using Accuracy, Precision, Recall and F1-score metrics and visualize using Confusion Matrix
- 7. Decision Boundary Visualization: Plot the non-linear decision boundary produced by the RBF SVM. Observe the flexible, curved boundary adapting to the moon-shaped clusters.

SVM - Practical Implementation

Step 1: Importing Libraries

Importing Libraries

```
# Numerical and data handling libraries
import numpy as np
import pandas as pd

# Visualization libraries
import matplotlib.pyplot as plt
import seaborn as sns
from mlxtend.plotting import plot_decision_regions

print("Loaded libraries")
```

Output:

Loaded libraries

Dataset utilities

```
from sklearn.datasets import load_wine, make_moons
# Preprocessing and model utilities
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
# Models
from sklearn.svm import SVC
# Evaluation metrics
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)

print("Loaded Sklearn libraries")
```

Output:

Loaded Sklearn libraries

Step 2: Loading Dataset

Loading and Reading Data

```
wine = load_wine()
wine_df = pd.DataFrame(wine.data, columns=wine.feature_names)

wine_df["target"] = wine.target
wine_df
```

Output:

	alco.	malic_a	ash	alcalinity_of.	magnes	total_phe.	flavano.	nonflavanoid_phe.	proanthocya.	color_inten.	hue	od280/od315_of_diluted_.	prol.	tar.
0	14.23	1.71	2.42	15.6	127.0	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065.	0
1	13.20	1.78	2.57	11.2	100.0	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050.	0
2	13.16	2.36	2.74	18.6	101.0	2.80	3.24	0.30	2.81	5.68	1.05	3.17	1185.	0
3	14.37	1.95	2.69	16.8	113.0	3.85	3.49	0.24	2.18	7.80	0.85	3.45	1480.	0
4	13.24	2.59	2.80	21.0	118.0	2.80	2.69	0.39	1.82	4.32	1.05	2.93	735.0	0
...														
170	13.71	5.65	2.42	20.5	95.0	1.68	0.61	0.52	1.06	7.70	0.85	1.74	740.0	2
171	13.40	3.91	2.42	23.0	102.0	1.80	0.75	0.43	1.41	7.30	0.85	1.56	750.0	2
172	13.27	4.28	2.42	20.0	120.0	1.59	0.69	0.43	1.35	10.20	0.85	1.56	835.0	2
173	13.17	2.59	2.42	20.0	120.0	1.65	0.68	0.53	1.46	9.30	0.85	1.62	840.0	2
177	14.13	4.10	2.42	24.5	96.0	2.05	0.76	0.56	1.35	9.20	0.85	1.60	560.0	2

Output:

178 rows x 14 columns

Display Wine dataset info

```
wine_df.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 178 entries, 0 to 177
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   alcohol                               178 non-null    float64
1   malic_acid                            178 non-null    float64
2   ash                                   178 non-null    float64
3   alcalinity_of_ash                     178 non-null    float64
4   magnesium                             178 non-null    float64
5   total_phenols                         178 non-null    float64
6   flavanoids                            178 non-null    float64
7   nonflavanoid_phenols                  178 non-null    float64
8   proanthocyanins                       178 non-null    float64
9   color_intensity                       178 non-null    float64
10  hue                                    178 non-null    float64
11  od280/od315_of_diluted_wines          178 non-null    float64
12  proline                               178 non-null    float64
13  target                                178 non-null    int32
dtypes: float64(13), int32(1)
memory usage: 19.0 KB
```

Generate two-moons dataset

```
X_moons, y_moons = make_moons(n_samples=700, noise=0.15, random_state=42)
moons_df = pd.DataFrame(X_moons, columns=["Feature_1", "Feature_2"])
moons_df["target"] = y_moons
moons_df
```

Output:

	Feature_1	Feature_2	target
0	-0.036777	1.017428	0
1	0.991181	-0.542893	1
2	0.135605	0.095587	1
3	0.380490	0.882966	0
4	-0.795374	0.193136	0
...			
695	1.147087	0.520498	0
696	0.200663	0.598349	0
697	-0.384952	0.547244	0
698	0.452570	-0.174473	1
699	0.534527	0.910810	0

Output:

700 rows x 3 columns

Step 3: Data Analysis

Data Analysis

```
wine_df = pd.concat(
    [wine_df[["flavanoids", "color_intensity"]], wine_df["target"]],
    axis=1
)

wine_target_names = {
    0: "Barolo",
    1: "Grignolino",
    2: "Barbera"
}

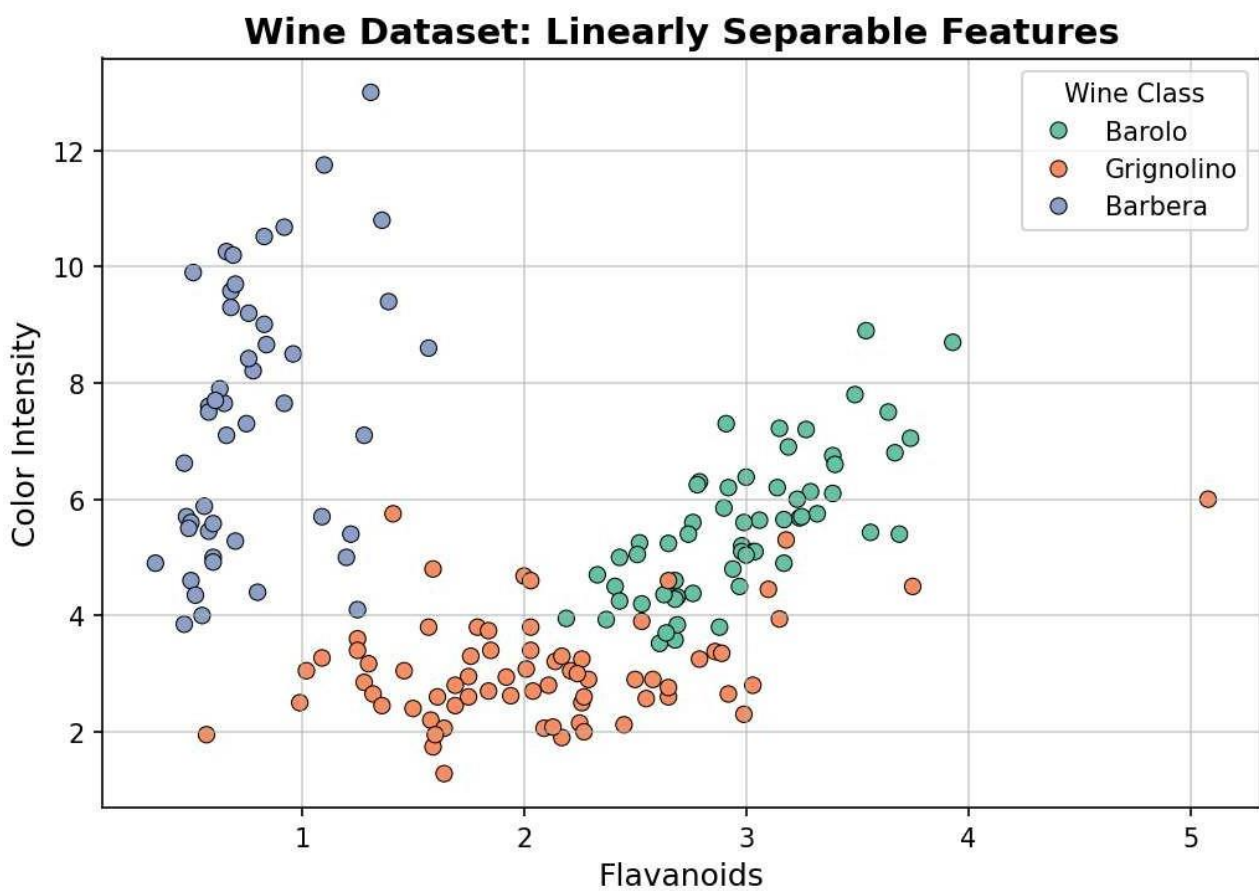
wine_df["class_name"] = wine_df["target"].map(wine_target_names)
wine_df[["target", "class_name"]].value_counts()
```

Output:

```
target  class_name
1      Grignolino    71
0      Barolo       59
2      Barbera      48
Name: count, dtype: int64
```

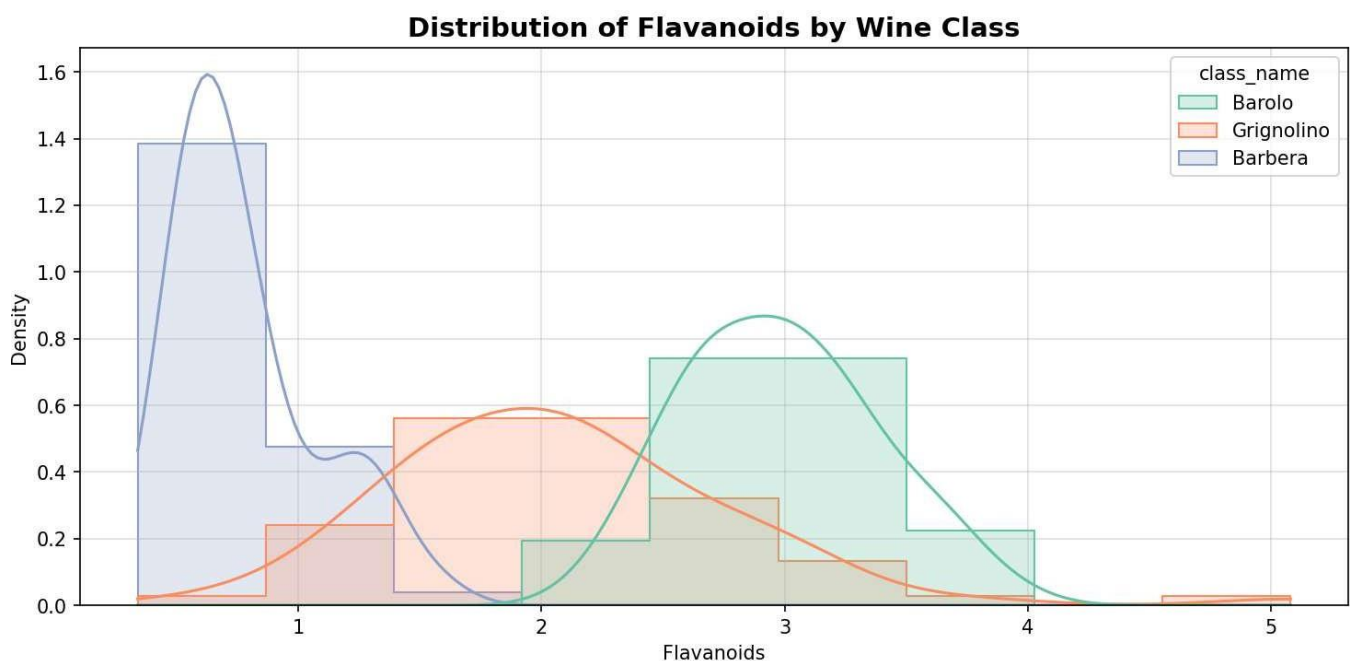
Visualizing Wine Dataset

```
plt.figure(figsize=(7, 5))
sns.scatterplot(
    data=wine_df,
    x="flavanoids",
    y="color_intensity",
    hue="class_name",
    palette="Set2",
    s=40,
    edgecolor="black"
)
plt.title("Wine Dataset: Linearly Separable Features", fontsize=14, fontweight="bold")
plt.xlabel("Flavanoids", fontsize=12)
plt.ylabel("Color Intensity", fontsize=12)
plt.legend(title="Wine Class")
plt.grid(alpha=0.6)
plt.show()
```



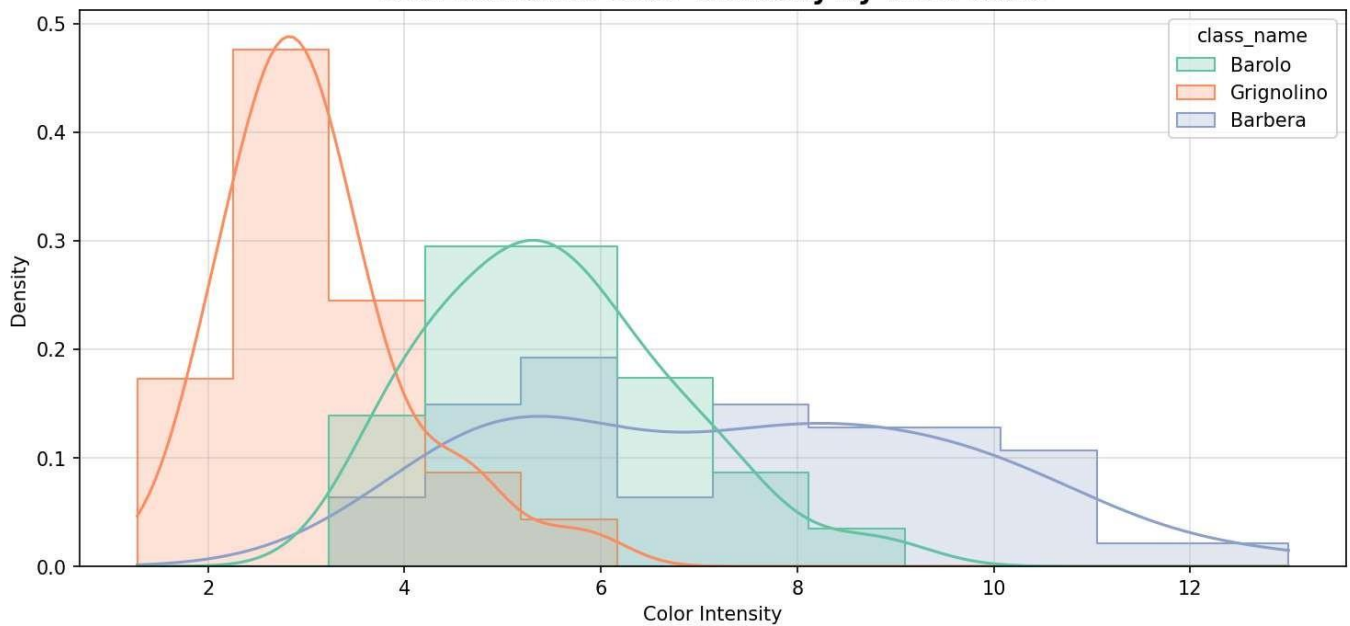
Histogram of Flavanoids

```
plt.figure(figsize=(10, 5))
sns.histplot(
    data=wine_df,
    x="flavanoids",
    hue="class_name",
    kde=True,
    palette="Set2",
    element="step",
    stat="density",
    common_norm=False
)
plt.title("Distribution of Flavanoids by Wine Class", fontsize=14, fontweight="bold")
plt.xlabel("Flavanoids")
plt.ylabel("Density")
plt.grid(alpha=0.4)
plt.show()
```

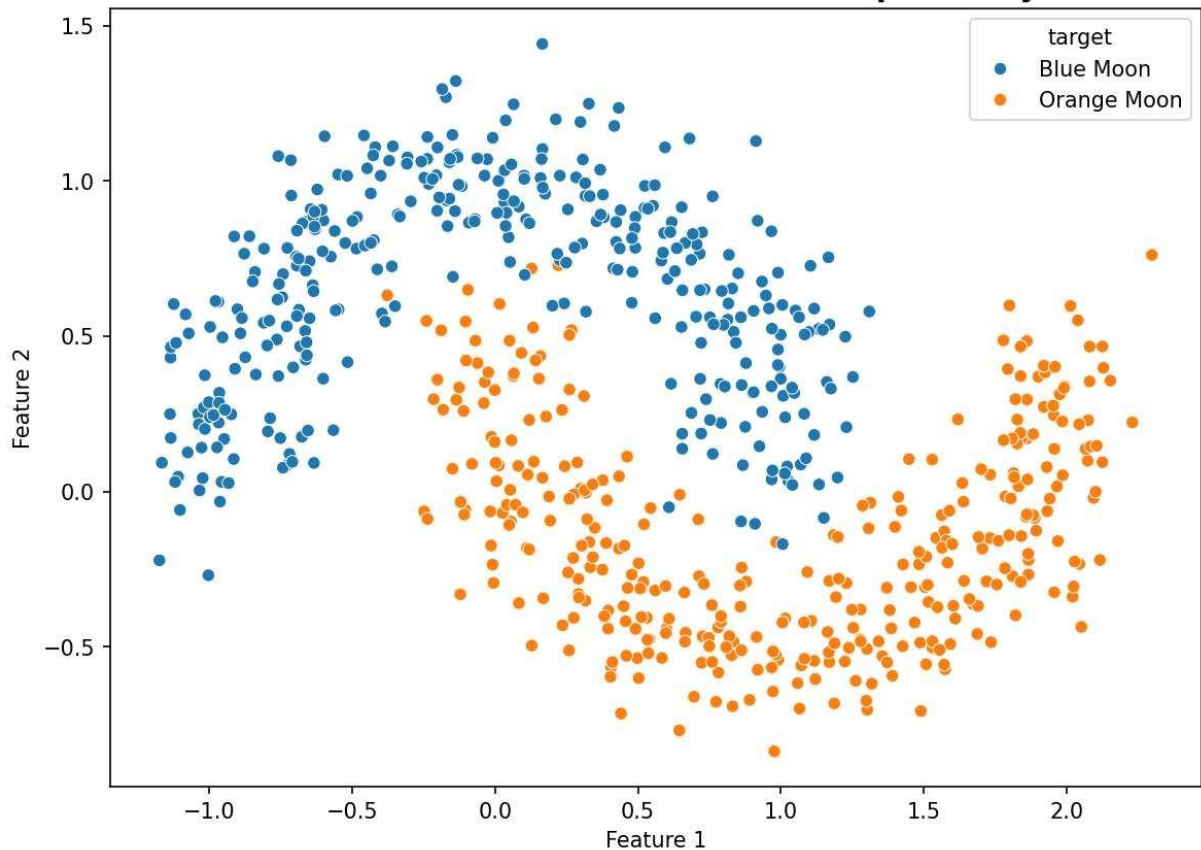


Histogram of Color Intensity

```
plt.figure(figsize=(10, 5))
sns.histplot(
    data=wine_df,
    x="color_intensity",
    hue="class_name",
    kde=True,
    palette="Set2",
    element="step",
    stat="density",
    common_norm=False
)
plt.title("Distribution of Color Intensity by Wine Class", fontsize=14, fontweight="bold")
plt.xlabel("Color Intensity")
plt.ylabel("Density")
plt.grid(alpha=0.4)
plt.show()
```

Distribution of Color Intensity by Wine Class**# Visualizing Two Moons Dataset**

```
plt.figure(figsize=(8,6))
sns.scatterplot(
    x=moons_df["Feature_1"],
    y=moons_df["Feature_2"],
    hue=moons_df["target"].map({0: "Blue Moon", 1: "Orange Moon"}),
    palette=["#1f77b4", "#ff7f0e"]
)
plt.title("Two Moons Dataset - Non-Linear Separability", fontsize=14, fontweight='bold')
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

Two Moons Dataset - Non-Linear Separability**Step 4: Data Preprocessing****# Data Preprocessing**

```
X_wine = wine_df[["flavanoids", "color_intensity"]]
y_wine = wine_df["target"]

Xw_train, Xw_test, yw_train, yw_test = train_test_split(
    X_wine, y_wine, test_size=0.2, random_state=42, stratify=y_wine
)
Xw_train.shape, Xw_test.shape, yw_train.shape, yw_test.shape
```

Output:

```
((142, 2), (36, 2), (142,), (36,))
```

Train/Test split for Two Moons dataset

```
Xm_train, Xm_test, ym_train, ym_test = train_test_split(
    X_moons, y_moons, test_size=0.2, random_state=42, stratify=y_moons
)
Xm_train.shape, Xm_test.shape, ym_train.shape, ym_test.shape
```

Output:

```
((560, 2), (140, 2), (560,), (140,))
```

Feature Scaling

```
scaler = StandardScaler()
scaler
```

Output:

```
StandardScaler()
```

Scale Wine training and test data

```
Xw_train_scaled = scaler.fit_transform(Xw_train)
Xw_test_scaled = scaler.transform(Xw_test)

print("Wine data scaled successfully!")
```

Output:

```
Wine data scaled successfully!
```

View original Wine training data (before scaling)

```
Xw_train[:5]
```

Output:

	flavanoids	color_intensity
10	2.81	5.40
149	3.17	5.75
90	3.00	5.50
59	2.91	5.60
147	0.66	9.39

View scaled Wine training data (after scaling)

```
Xw_train_scaled[:5]
```

Output:

```
array([[ 0.73229212, -0.16746725],
       [ 1.33318146,  0.30530313],
       [ 1.006382   , -0.081509   ],
       [ 0.81662747,  0.262324   ],
       [-1.29175618,  1.47433535]])
```

Scale Two Moons training and test data

```
Xm_train_scaled = scaler.fit_transform(Xm_train)
Xm_test_scaled = scaler.transform(Xm_test)

print("Two Moons data scaled successfully!")
```

Output:

```
Two Moons data scaled successfully!
```


View original Two Moons training data (before scaling)

```
Xm_train[:5]
```

Output:

```
array([[ 1.3723603 , -0.55081882],
       [ 0.63613117,  0.78345978],
       [-0.43531115,  0.96004336],
       [ 0.41718644,  1.17721089],
       [-1.13531808,  0.17220389]])
```

View scaled Two Moons training data (after scaling)

```
Xm_train_scaled[:5]
```

Output:

```
array([[ 1.01466867, -1.57697415],
       [ 0.17320777,  1.01862443],
       [-1.05137949,  1.36213595],
       [-0.07703149,  1.78459625],
       [-1.85144083, -0.17046376]])
```

Step 5: Model Training

Model Training

```
# Training linear Kernel SVM in wine dataset with 2 features
svm_linear_wine = SVC(kernel="linear")
svm_linear_wine.fit(Xw_train_scaled, yw_train)
svm_linear_wine
```

Output:

```
SVC(kernel='linear')
```

Training SVMs with linear and RBF kernels on two-moons toy dataset

```
svm_linear_moon = SVC(kernel="linear")
svm_rbf_moon = SVC(kernel="rbf")

svm_linear_moon.fit(Xm_train_scaled, ym_train)
svm_rbf_moon.fit(Xm_train_scaled, ym_train)
print("Model trained")
```

Output:

```
Model trained
```

Step 6: Model Evaluation

Model Evaluation

```
# Wine dataset case
wine_preds = svm_linear_wine.predict(Xw_test_scaled)
wine_preds
```

Output:

```
array([0, 2, 0, 0, 1, 0, 0, 0, 1, 2, 1, 2, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1,
       0, 0, 0, 1, 0, 2, 1, 2, 0, 2, 1, 2, 2, 2])
```

Wine dataset - Classification metrics

```
print("Accuracy:", accuracy_score(yw_test, wine_preds))
print("Precision:", precision_score(yw_test, wine_preds, average="macro"))
print("Recall:", recall_score(yw_test, wine_preds, average="macro"))
print("F1 Score:", f1_score(yw_test, wine_preds, average="macro"))

print("\n\nClassification Report:\n\n")
print(classification_report(yw_test, wine_preds))
```

Output:

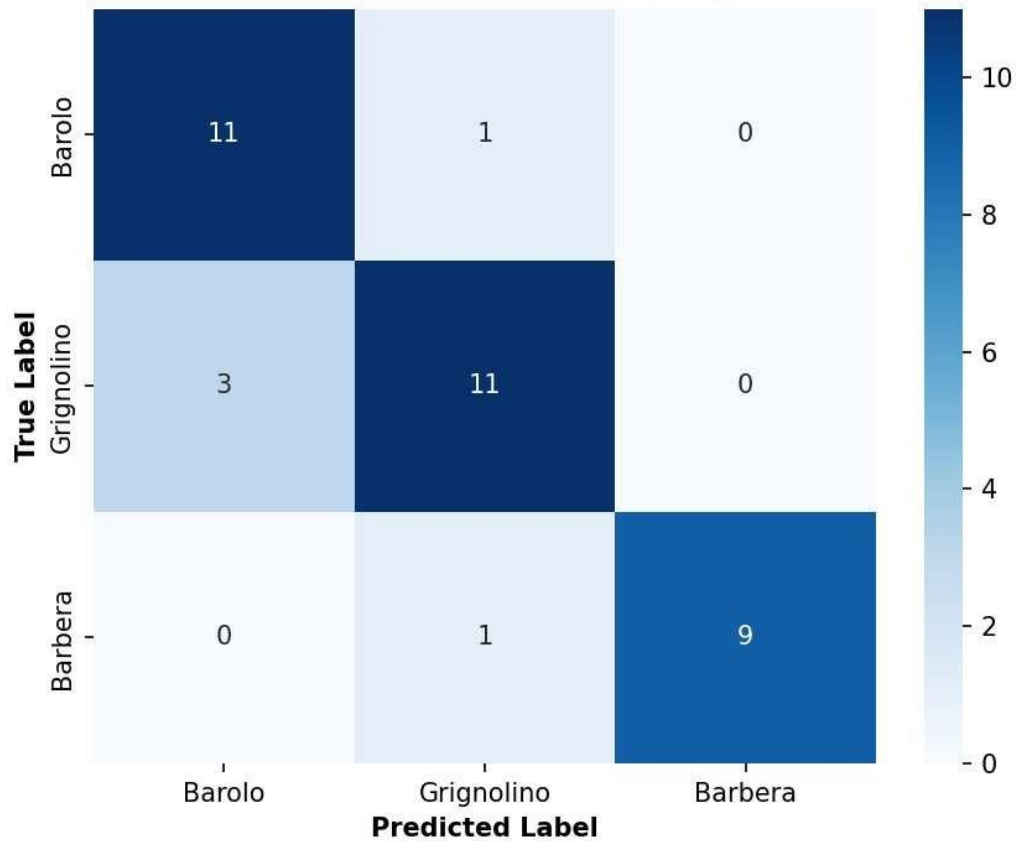
```
Accuracy: 0.8611111111111112
Precision: 0.8772893772893773
Recall: 0.8674603174603175
F1 Score: 0.8694456940070975
Classification Report:
precision    recall  f1-score   support
0           0.79      0.92      0.85         12
1           0.85      0.79      0.81         14
2           1.00      0.90      0.95         10
accuracy                   0.86         36
macro avg              0.88      0.87      0.87         36
weighted avg              0.87      0.86      0.86         36
```

Confusion Matrix

```
labels = list(wine_target_names.keys())
class_names = list(wine_target_names.values())

plt.figure(figsize=(6, 5))
sns.heatmap(
    confusion_matrix(yw_test, wine_preds, labels=labels),
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=class_names,
    yticklabels=class_names
)

plt.title("Wine Dataset (Linear Kernel)", fontweight='bold')
plt.xlabel("Predicted Label", fontweight='bold')
plt.ylabel("True Label", fontweight='bold')
plt.tight_layout()
plt.show()
```

Wine Dataset (Linear Kernel)**# Two moon dataset case**

```
moon_linear_preds = svm_linear_moon.predict(Xm_test_scaled)
moon_linear_preds
```

Output:

```
array([0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0,
0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1,
1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1,
0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0,
1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 0,
1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0,
0, 0, 1, 0, 0, 1, 1, 1])
```

Confusion Matrix

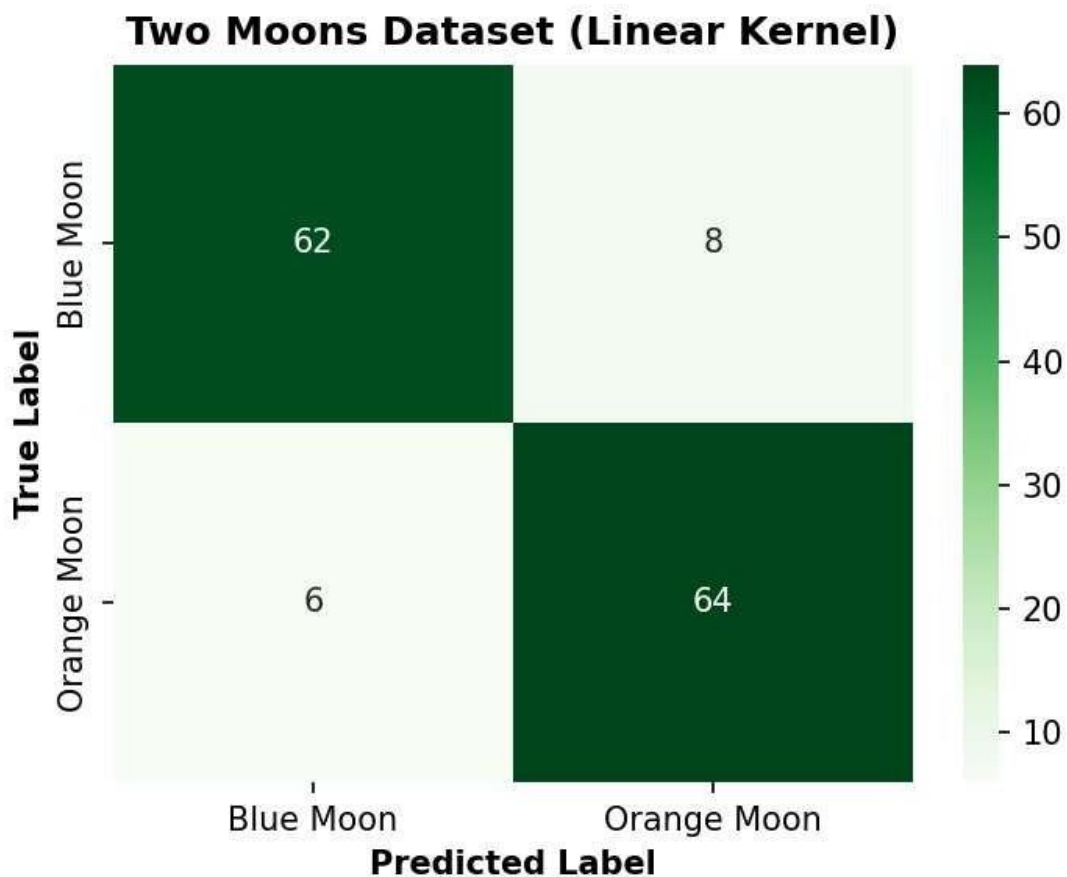
```

class_names = ['Blue Moon', 'Orange Moon']

plt.figure(figsize=(5, 4))
sns.heatmap(
    confusion_matrix(ym_test, moon_linear_preds),
    annot=True,
    fmt="d",
    cmap="Greens",
    xticklabels=class_names,
    yticklabels=class_names
)

plt.title("Two Moons Dataset (Linear Kernel)", fontweight='bold')
plt.xlabel("Predicted Label", fontweight='bold')
plt.ylabel("True Label", fontweight='bold')
plt.tight_layout()
plt.show()

```



Two Moons - Linear kernel classification metrics

```

print("Accuracy:", accuracy_score(ym_test, moon_linear_preds))
print("Precision:", precision_score(ym_test, moon_linear_preds, average="macro"))
print("Recall:", recall_score(ym_test, moon_linear_preds, average="macro"))
print("F1 Score:", f1_score(ym_test, moon_linear_preds, average="macro"))

print("\n\nClassification Report:\n\n")
print(classification_report(ym_test, moon_linear_preds))

```

Output:

```

Accuracy: 0.9
Precision: 0.9003267973856208
Recall: 0.8999999999999999
F1 Score: 0.8999795876709533
Classification Report:
precision    recall  f1-score   support

0           0.91      0.89      0.90         70
1           0.89      0.91      0.90         70

accuracy          0.90         140
macro avg         0.90      0.90      0.90         140
weighted avg      0.90      0.90      0.90         140

```

Two moon dataset case

```

moon_rbf_preds = svm_rbf_moon.predict(Xm_test_scaled)
moon_rbf_preds

```

Output:

```

array([0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0,
0, 0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1,
1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1,
0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0,
1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 0,
1, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0,
1, 1, 1, 0, 1, 1, 0, 1])

```

Two Moons - RBF Kernel Confusion Matrix

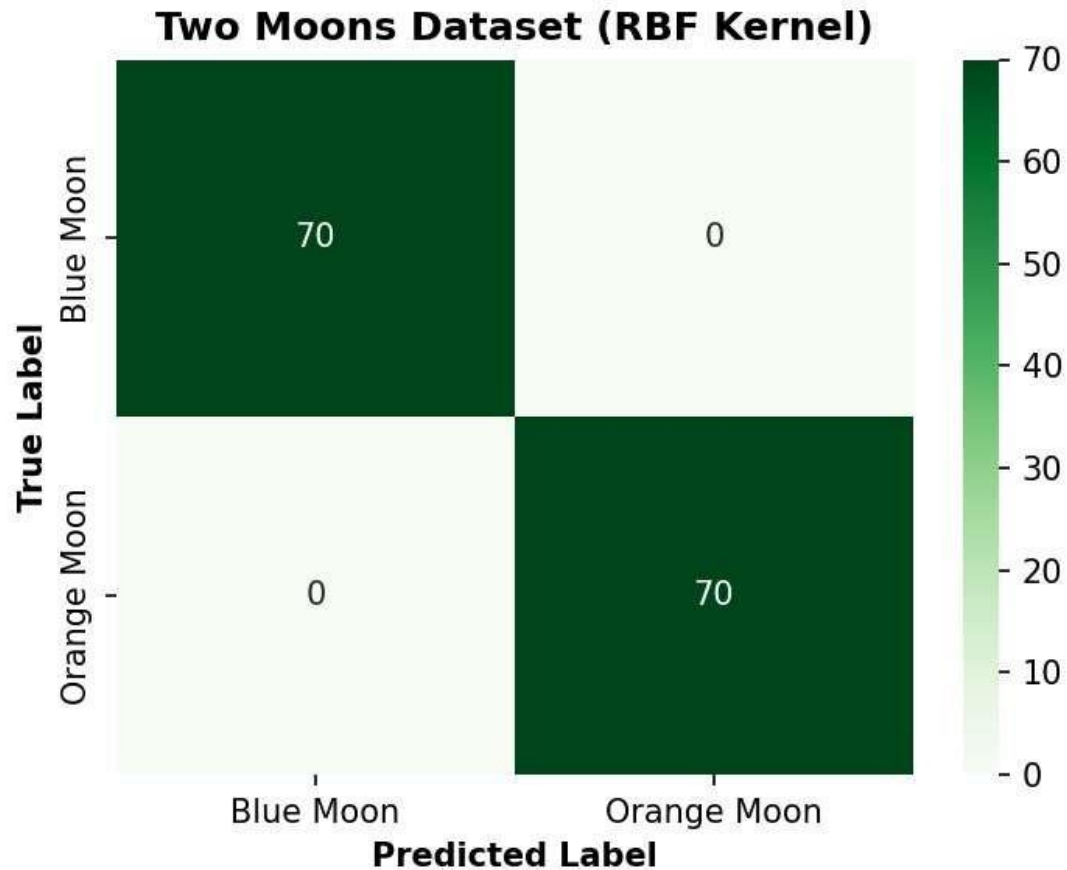
```

class_names = ['Blue Moon', 'Orange Moon']

plt.figure(figsize=(5, 4))
sns.heatmap(
    confusion_matrix(ym_test, moon_rbf_preds),
    annot=True,
    fmt="d",
    cmap="Greens",
    xticklabels=class_names,
    yticklabels=class_names
)

plt.title("Two Moons Dataset (RBF Kernel)", fontweight='bold')
plt.xlabel("Predicted Label", fontweight='bold')
plt.ylabel("True Label", fontweight='bold')
plt.tight_layout()
plt.show()

```



Two Moons - RBF kernel classification metrics (Perfect accuracy!)

```
print("Accuracy:", accuracy_score(ym_test, moon_rbf_preds))
print("Precision:", precision_score(ym_test, moon_rbf_preds, average="macro"))
print("Recall:", recall_score(ym_test, moon_rbf_preds, average="macro"))
print("F1 Score:", f1_score(ym_test, moon_rbf_preds, average="macro"))

print("\n\nClassification Report:\n\n")
print(classification_report(ym_test, moon_rbf_preds))
```

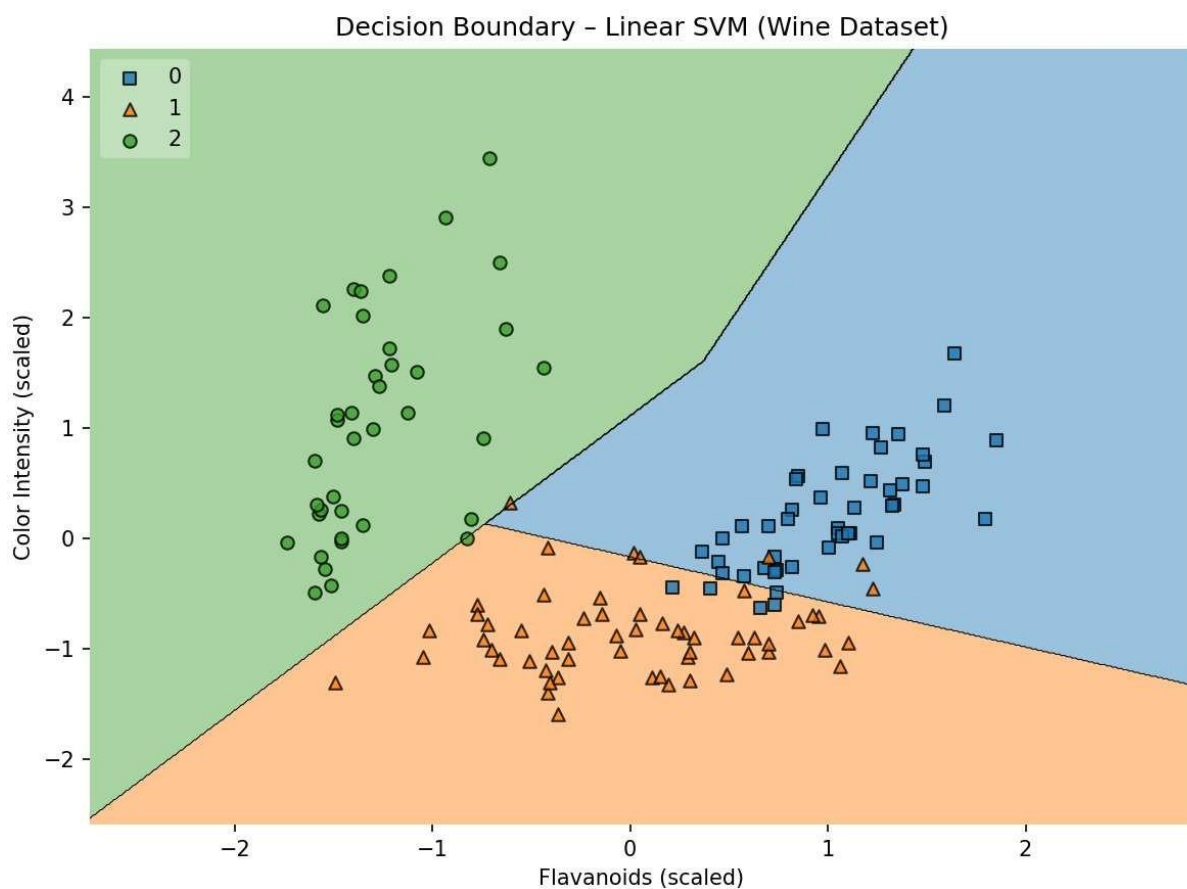
Output:

```
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0
Classification Report:
precision    recall  f1-score   support
0           1.00     1.00     1.00        70
1           1.00     1.00     1.00        70
accuracy                   1.00       140
macro avg           1.00     1.00     1.00       140
weighted avg           1.00     1.00     1.00       140
```

Step 7: Model Simulation

Model testing

```
# Decision Boundaries - Wine
plot_decision_regions(
    Xw_train_scaled,
    yw_train.values,
    clf=svm_linear_wine,
    legend=2
)
plt.title("Decision Boundary - Linear SVM (Wine Dataset)")
plt.xlabel("Flavanoids (scaled)")
plt.ylabel("Color Intensity (scaled)")
plt.show()
```



Decision Boundaries - Two Moons

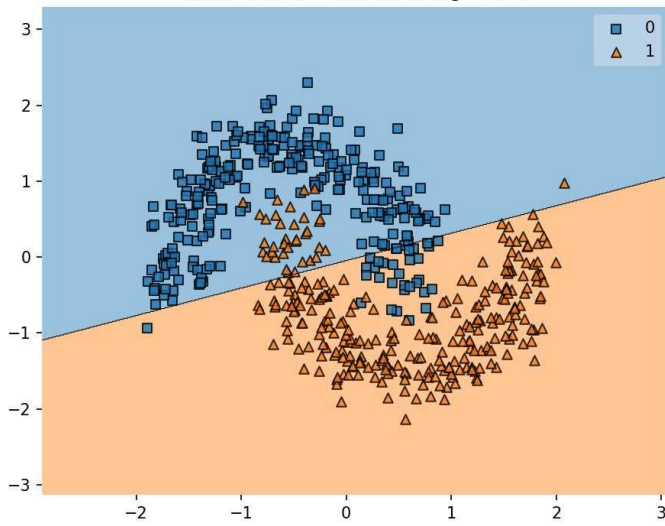
```
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plot_decision_regions(Xm_train_scaled, ym_train, clf=svm_linear_moon)
plt.title("Linear Kernel - Blue vs Orange Moon")

plt.subplot(1,2,2)
plot_decision_regions(Xm_train_scaled, ym_train, clf=svm_rbf_moon)
plt.title("RBF Kernel - Blue vs Orange Moon")

plt.show()
```

Linear Kernel - Blue vs Orange Moon



RBF Kernel - Blue vs Orange Moon

